

ATHARV VERMA

verma.atharv@gmail.com • +91 9599492763 • [LinkedIn](#) • [GitHub](#) • [LeetCode](#)

PROFESSIONAL SUMMARY

Computer Science undergraduate with a strong foundation in Data Structures, Algorithms, Object-Oriented Programming, and software engineering principles. Experienced across the software development lifecycle — software design, feature development, testing, and debugging — with a focus on scalable, maintainable architecture and systematic problem solving. Maintains a perfect 10.0 CGPA in Semester 2 while actively building production-style projects and pursuing competitive technical internships.

EDUCATION

SRM Institute of Science and Technology

2025 – 2029 (Expected)

B.Tech, Computer Science and Engineering (CSE Core) — CGPA: Sem 1: 9.7 | Sem 2: 10.0 (Perfect GPA)

TECHNICAL SKILLS

- ▶ Languages: C, C++, Java, JavaScript (ES6+), HTML5, CSS3
- ▶ Core CS Fundamentals: Object-Oriented Software Design (Inheritance, Encapsulation), Data Structures & Algorithms, Memory Management, Pointers, Recursion, Time/Space Complexity Analysis, Performance Optimization
- ▶ Engineering Practice: Git, GitHub, VS Code, Chrome DevTools, Debugging, Software Development Lifecycle (SDLC), Manual/Functional Testing, Version Control, Technical Documentation
- ▶ Web Technologies: HTML5, CSS3, JavaScript (ES6+), Responsive Design
- ▶ Currently Building: Advanced DSA in C++, Full-Stack Web Development (Node.js/React)

ALGORITHMIC PROBLEM SOLVING

LeetCode — leetcode.com/u/atharv5492

- ▶ Apply a structured problem-solving methodology across arrays, hashing, and bit manipulation — systematically decomposing each problem, deriving a brute-force baseline, then iteratively refining toward optimal time/space complexity using targeted DSA techniques.
- ▶ Maintain a disciplined daily practice cadence while advancing through Advanced DSA in C++, translating coursework theory (recursion, pointer arithmetic, memory management, complexity analysis) into working, tested implementations.

PROJECTS

Nexa Bank — Banking Platform Front-End Architecture *HTML5 · CSS3 · JavaScript (ES6+)*

[Live Demo](#)

[GitHub](#)

- ▶ Designed and developed the architectural backbone of a multi-page banking platform in vanilla JavaScript, building a stateful transaction simulation engine that propagates balance updates consistently across concurrent account views — laying the groundwork for the in-progress full-stack (backend + persistent database) upgrade for improved scalability.
- ▶ Built a component-driven, semantic HTML5/modular CSS3 layer to decouple presentation from business logic for long-term maintainability, achieving pixel-consistent rendering across 3+ browser engines and debugging layout-breaking edge cases typical of unstructured front-end builds.

Banking Management System *C++ · Object-Oriented Design*

[GitHub](#)

- ▶ Designed, developed, and debugged a terminal-based banking engine to solve a core extensibility problem: unifying divergent account behaviors without duplicating transaction logic, via a C++ inheritance hierarchy with a shared base class.
- ▶ Structured the class hierarchy for maintainability and straightforward feature development — onboarding a new account type requires extending a single subclass rather than modifying existing transaction code, reflecting open/closed-principle engineering discipline.
- ▶ Performed manual functional testing across core transaction flows to verify balance accuracy and product quality, catching edge-case failures before finalizing the module.

College ERP System *C++ · Object-Oriented Design*

[GitHub](#)

- ▶ Developed a multi-student academic ERP addressing a defined user requirement — attendance and grade administration at scale — using dynamically managed STL vectors to support an arbitrary number of student records with automated 75%-threshold attendance enforcement and grade computation.
- ▶ Engineered an ID-based authentication layer with real-time logout on repeated credential failure, improving system reliability by hardening against unauthorized access while preserving persistent, session-wide record integrity.

Library Management System *C++ · Object-Oriented Design*

[GitHub](#)

- ▶ Designed a dual-portal architecture (Admin & Student) based on distinct user requirements for each role, enforcing role-based access control so each user class is strictly confined to actions within its permission scope — a separation-of-concerns pattern common in production multi-tenant systems.
- ▶ Implemented a 3-attempt account lockout mechanism alongside full book inventory and issue/return tracking, improving reliability and product quality by reducing unauthorized-access risk while streamlining end-to-end circulation management.

CERTIFICATIONS

- ▶ GenAI Powered Data Analytics Job Simulation — Tata · The Forage (June 2026): EDA, financial risk profiling, and AI-driven delinquency prediction for collections strategy.
- ▶ Career Essentials in Generative AI — Microsoft · LinkedIn Learning (June 2026): GenAI fundamentals, Copilot, Responsible AI governance.
- ▶ AI Tools & ChatGPT Workshop — be10X (June 2026): High-speed AI workflows for presentations, data analysis, and AI-assisted debugging.